

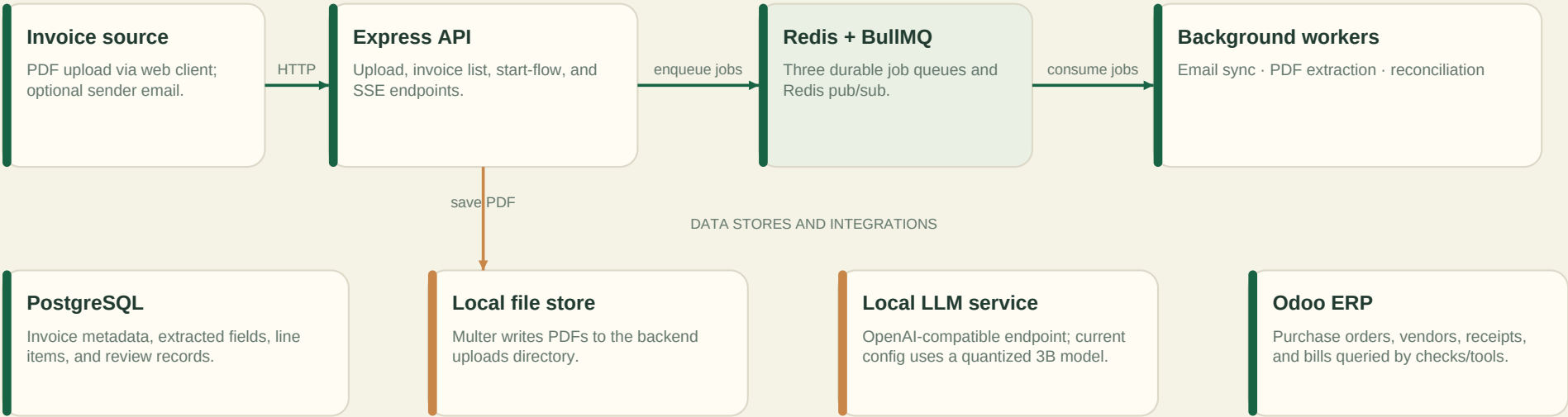
Invoice review, from PDF to decision

A queue-based invoice processing system with deterministic ERP checks, a read-only investigation agent, and a live review interface.

DESIGN PRINCIPLE

Run predictable business rules first. Use the local language model only when an exception needs investigation. Keep ERP investigation read-only; reserve final action for the policy engine and human review.

SYSTEM MAP



CURRENT

Local disk for PDF storage · local model endpoint · Odoo integration · live SSE updates

PLANNED

Object/blob storage in production · Slack notifications/integration under development

01 / INTAKE + EXTRACTION

Turn an uploaded PDF into invoice data

The upload endpoint stores the file locally and queues metadata. Workers process the file asynchronously so the request can return quickly.

01 Upload

The Express API accepts a PDF at **POST /upload**. Multer writes it to the local **uploads/** directory and assigns a unique filename. Current limit: 10 MB.

02 Queue intake metadata

The API enqueues filename, size, local path, and supplied sender email in BullMQ's **email-sync** queue.

03 Create invoice record

The email-sync worker creates the initial PostgreSQL invoice row with file metadata and initial status, then publishes the received event.

04 Start processing

The UI fetches invoices through **GET /invoice**. A reviewer starts a flow with **POST /startFlow/:invoiceId**; the API reads file metadata from PostgreSQL and queues the extraction job.

05 Parse and extract

The PDF worker reads the local file, extracts text with the PDF parser, and sends text to the configured local LLM using an OpenAI-compatible API.

06 Persist and hand off

The worker publishes extracted invoice fields, writes the structured fields and line items to PostgreSQL, then sends the extracted invoice to the reconciliation queue.

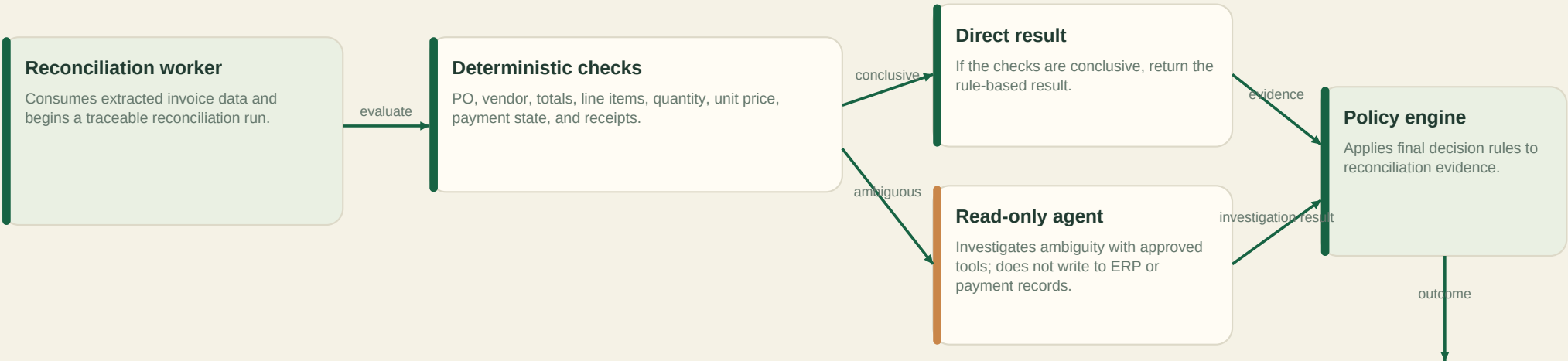
MODEL BOUNDARY

The current extraction config uses a quantized local Llama 3.2 3B Instruct model. Validate its structured output and retain the source PDF for audit.

02 / RECONCILIATION + DECISION

Rules first. Investigation only when needed.

The reconciliation worker evaluates the extracted invoice against ERP data. Straightforward cases follow deterministic business logic; ambiguous cases invoke a constrained agent loop.



AGENT TOOL BOUNDARY

The agent receives the deterministic mismatch and invoice context. Its controller exposes read-only Odoo tools for purchase orders, vendor details, order lines, receipts, and bills. Tool results are evidence; the agent cannot edit ERP records or issue payments.

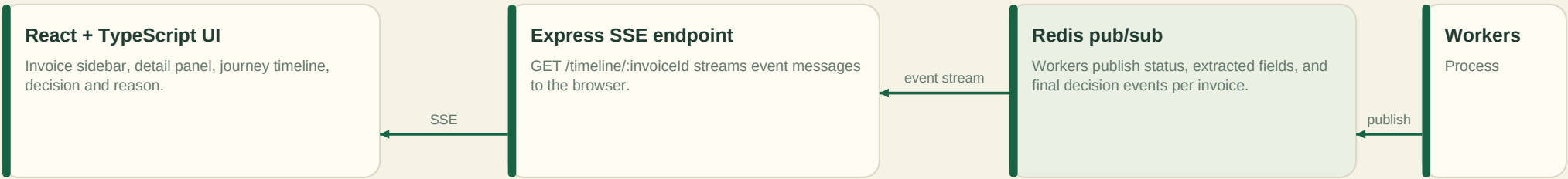
POLICY OUTCOMES

- READY_FOR_PAYMENT** Checks pass and payment may proceed.
- REVIEW_REQUIRED** A human should resolve remaining ambiguity.
- BLOCKED** A business rule prevents processing.

Odoo is the current ERP integration. The deterministic path and agent tools both read ERP data; the policy engine combines the findings into the result used by the application.

A live review surface with clear system boundaries

The UI combines persisted invoice records with per-invoice events. The design separates current local development behavior from the production direction.



EVENTS SHOWN LIVE

The browser listens on an invoice-specific stream. Status messages advance the hand-drawn journey path. Extraction fills the supplier, invoice number, amount, and purchase order; completion carries the policy decision and reason.

PERSISTENCE

PostgreSQL stores invoice metadata and extracted fields, line items, event history, reconciliation results, and agent runs. The UI loads invoice records through the API; SSE keeps the selected invoice current while work runs.

CURRENT VS. NEXT

CURRENT MVP

- Files: local disk
- Queue / events: BullMQ and Redis
- Database: PostgreSQL via Drizzle
- LLM: local OpenAI-compatible endpoint
- ERP: Odoo read integrations
- Notifications: live browser updates via SSE

PRODUCTION DIRECTION

Replace local PDF paths with object/blob storage and durable object references. Scale API and workers independently; add authentication, authorization, secrets management, retention policies, and operational monitoring. Add Slack notifications after approval and retry behavior are defined. Preserve the read-only agent boundary unless a separate, auditable human-approved write path is designed.